

Ollama

WangFei

2026 年 3 月 5 日

目录

前言	i
第一卷 基础篇	1
第 1 篇 Ollama 简介	3
1.1 什么是 Ollama	3
1.2 Ollama 的特点与优势	3
1.3 Ollama 的应用场景	3
第 2 篇 Ollama 安装与配置	5
2.1 Windows 系统安装	5
2.2 macOS 系统安装	5
2.3 Linux 系统安装	5
2.4 基本配置选项	5
第 3 篇 Ollama 命令行工具	7
3.1 基本命令使用	7
3.2 模型管理	7
3.3 参数配置	7
第二卷 实战篇	9
第 4 篇 模型部署与使用	11
4.1 常用模型介绍	11
4.2 模型下载与管理	11
4.3 模型推理与对话	11
4.3.1 OpenClaw 开源 AI 助手	11
第 5 篇 Ollama API 开发	13
5.1 REST API 使用	13

5.1.1	Streaming 功能	13
5.2	SDK 集成	15
5.3	示例代码	15
第 6 篇	Ollama 与其他工具集成	17
6.1	与 LangChain 集成	17
6.2	与 LLM 框架集成	17
6.3	与应用系统集成	17
6.3.1	运行任何应用或代理	17
第三卷	高级篇	19
第 7 篇	Ollama 架构与原理	21
7.1	系统架构	21
7.2	模型加载机制	21
7.3	推理优化技术	21
第 8 篇	自定义模型开发	23
8.1	模型微调	23
8.2	模型量化	23
8.3	模型部署最佳实践	23
第 9 篇	Ollama 性能优化	25
9.1	硬件资源配置	25
9.2	推理速度优化	25
9.3	内存使用优化	25

前言

第一卷

基础篇

第 1 篇 Ollama 简介

1.1 什么是 Ollama

Ollama 是使用大型语言模型（如 gpt-oss、Gemma 3、DeepSeek-R1、Qwen3 等）的最简单方法。

1.2 Ollama 的特点与优势

1.3 Ollama 的应用场景

第 2 篇 Ollama 安装与配置

2.1 Windows 系统安装

在 PowerShell 中运行以下命令即可安装 Ollama:

```
irm https://ollama.com/install.ps1 | iex
```

或者访问 <https://ollama.com/download> 下载 Ollama 安装程序。

2.2 macOS 系统安装

2.3 Linux 系统安装

2.4 基本配置选项

第 3 篇 Ollama 命令行工具

3.1 基本命令使用

Ollama 可在 macOS、Windows 和 Linux 上使用。在终端中运行 ‘ollama’ 打开交互式菜单：

```
ollama
```

使用 ↑/↓ 导航，按 enter 启动，→ 更改模型，esc 退出。
菜单提供快速访问：

- Run a model - 开始交互式聊天
- Launch tools - Claude Code、Codex、OpenClaw 等
- Additional integrations - 在”More...” 下可用

3.2 模型管理

3.3 参数配置

第二卷

实战篇

第 4 篇 模型部署与使用

4.1 常用模型介绍

4.2 模型下载与管理

4.3 模型推理与对话

使用 Ollama 启动开源模型进行编码工作，支持 Claude Code、Codex、OpenCode 等多种模型：

```
ollama launch claude  
ollama launch codex  
ollama launch opencode
```

这将启动由最佳开源模型提供支持的编码环境。

4.3.1 OpenClaw 开源 AI 助手

OpenClaw 是一个具有 100+ 技能的个人 AI 助手，能够自动化工作、回答问题并处理任务，由开源模型提供支持：

```
ollama launch openclaw
```


第 5 篇 Ollama API 开发

5.1 REST API 使用

使用 API 将 Ollama 集成到您的应用程序中：

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [{ "role": "user", "content": "Hello!" }]
}'
```

5.1.1 Streaming 功能

Streaming 允许您在模型生成文本时实时渲染文本。

- REST API 默认启用 Streaming，但 SDK 默认禁用
- 在 SDK 中启用 Streaming，需要将 stream 参数设置为 True

关键 Streaming 概念

1. **Chatting:** 流式传输部分助手消息。每个块包含内容，因此您可以在消息到达时渲染它们。
2. **Thinking:** 支持思考的模型会在每个块中发出 thinking 字段。在流式块中检测此字段，以在最终答案到达前显示或隐藏推理轨迹。
3. **Tool calling:** 监视每个块中的 tool_calls，执行请求的工具，并将工具输出附加回对话。

处理流式块

需要累积部分字段以维护对话历史。这对于工具调用特别重要，因为模型的思考、工具调用和执行的工具结果必须在下次请求中传递回模型。

以下是处理流式块的 Python 代码示例：

```
from ollama import chat

stream = chat(
    model='qwen3',
    messages=[{'role': 'user', 'content': 'What is 17 × 23?'}],
    stream=True,
)

in_thinking = False
content = ''
thinking = ''
for chunk in stream:
    if chunk.message.thinking:
        if not in_thinking:
            in_thinking = True
            print('Thinking:\n', end='', flush=True)
        print(chunk.message.thinking, end='', flush=True)
        # accumulate the partial thinking
        thinking += chunk.message.thinking
    elif chunk.message.content:
        if in_thinking:
            in_thinking = False
            print('\n\nAnswer:\n', end='', flush=True)
        print(chunk.message.content, end='', flush=True)
        # accumulate the partial content
        content += chunk.message.content

# append the accumulated fields to the messages for the next request
new_messages = [{ role: 'assistant', thinking: thinking, content: content }]
```

以下是处理流式块的 JavaScript 代码示例：

```
import ollama from 'ollama'

async function main() {
    const stream = await ollama.chat({
        model: 'qwen3',
        messages: [{ role: 'user', content: 'What is 17 × 23?' }],
```

```
    stream: true,
  })

  let inThinking = false
  let content = ''
  let thinking = ''

  for await (const chunk of stream) {
    if (chunk.message.thinking) {
      if (!inThinking) {
        inThinking = true
        process.stdout.write('Thinking:\n')
      }
      process.stdout.write(chunk.message.thinking)
      // accumulate the partial thinking
      thinking += chunk.message.thinking
    } else if (chunk.message.content) {
      if (inThinking) {
        inThinking = false
        process.stdout.write('\n\nAnswer:\n')
      }
      process.stdout.write(chunk.message.content)
      // accumulate the partial content
      content += chunk.message.content
    }
  }

  // append the accumulated fields to the messages for the next request
  new_messages = [{ role: 'assistant', thinking: thinking, content: content }]
}

main().catch(console.error)
```

5.2 SDK 集成

5.3 示例代码

第 6 篇 Ollama 与其他工具集成

6.1 与 LangChain 集成

6.2 与 LLM 框架集成

6.3 与应用系统集成

6.3.1 运行任何应用或代理

使用 Ollama，您可以将最新的开源模型连接到您喜欢的应用程序或代理，轻松
在它们之间切换：

```
ollama
```

这使得您可以灵活地为不同的任务选择最适合的模型，而无需修改应用程序代
码。

第三卷

高级篇

第 7 篇 Ollama 架构与原理

7.1 系统架构

7.2 模型加载机制

7.3 推理优化技术

第 8 篇 自定义模型开发

8.1 模型微调

8.2 模型量化

8.3 模型部署最佳实践

第 9 篇 Ollama 性能优化

9.1 硬件资源配置

9.2 推理速度优化

9.3 内存使用优化

